

Lecture 12

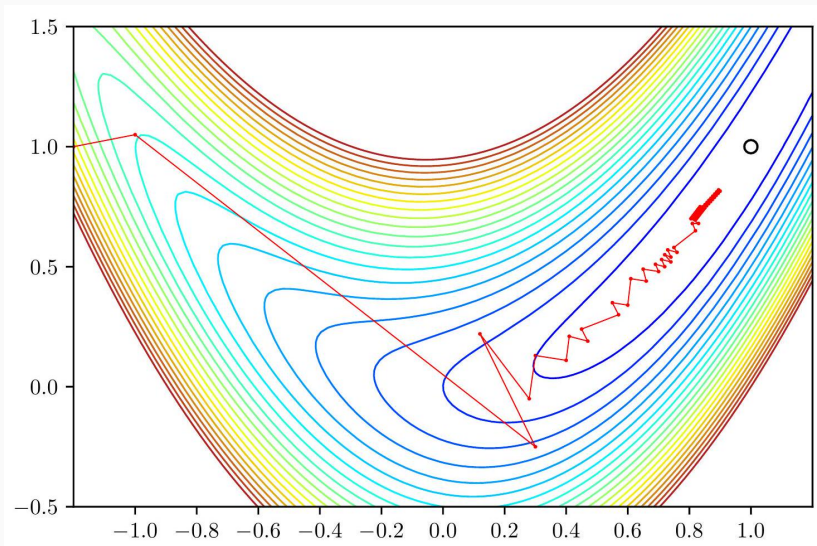
Kendra Burbank

Line search algorithms

OK, we have arrived at the point where we can actually talk about optimization algorithms!

We will suppose that we are minimizing a function f . Whether f is convex or not, the algorithm will be the same.

Basic idea: at each iteration, we pick a **direction** and decide **how far** to move in that direction. We move, and then rinse, lather, and repeat!



Formally,

1. Choose a starting point $\mathbf{x}_0 \in \mathbb{R}^n$ or $X_0 \in \mathbb{S}^n$
2. Choose a search direction $\mathbf{p}_k \in \mathbb{R}^n$ or $P_k \in \mathbb{S}^n$
3. Choose a step size/length $\alpha_k \in \mathbb{R}$.
4. Do $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$ or $X_{k+1} = X_k + \alpha_k P_k$
5. Repeat items 2 – 4 until stopping condition is met

Two things we need to find at each step:

1. Search direction p_k
2. Step size α_k

Finding the search direction

If f is not differentiable, use non-differentiable optimization techniques.

If f is differentiable, we can use first or second order approximations from Taylor's theorem:

$$f(\mathbf{x}_k + \alpha_k \mathbf{p}_k) \approx f(\mathbf{x}_k) + \alpha_k \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k \quad f(\mathbf{x}_k + \alpha_k \mathbf{p}_k) \approx f(\mathbf{x}_k) + \alpha_k \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k + \frac{1}{2} \alpha_k^2 \mathbf{p}_k^\top \nabla^2 f(\mathbf{x}_k) \mathbf{p}_k$$

First order approximations are used in: - steepest descent - conjugate gradient methods

Second order approximations are used in: - Newton's method - interior point methods (IPM) - quasi-Newton methods

Why not always use second order methods?

Second order methods require the computation of the Hessian, which is $O(n^2)$. In contrast, evaluating a gradient is only $O(n)$.

We start by describing some approaches for computing α_k and $\mathbf{p}_k$.

But we'll start simple and assume $\alpha_k = 1$ for all $k = 0, 1, \dots$

Using the linear approximation, we find the direction which reduces the function value the most:

$$\min_{\|\mathbf{p}\|=1} \left(f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top \mathbf{p} \right).$$

Note that we needed to set a limit on the length of $\mathbf{p}$ – otherwise the minimum would be negative infinity anytime the gradient is not 0 .

$$\min_{\|\mathbf{p}\|=1} \left(f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top \mathbf{p} \right).$$

By Cauchy-Schwarz,

$$-\|\nabla f(\mathbf{x}_k)\| \|\mathbf{p}\| \leq \nabla f(\mathbf{x}_k)^\top \mathbf{p} \leq \|\nabla f(\mathbf{x}_k)\| \|\mathbf{p}\|.$$

From this, we see that the minimum is attained when

$$\mathbf{p}_k = \frac{-\nabla f(\mathbf{x}_k)}{\|\nabla f(\mathbf{x}_k)\|}$$

$$\mathbf{p}_k = \frac{-\nabla f(\mathbf{x}_k)}{\|\nabla f(\mathbf{x}_k)\|}$$

But we are just trying to find a search *direction*. The magnitude is going to be accounted for in the step size α_k anyway. In other words, we can simply use

$$\mathbf{p}_k = -\nabla f(\mathbf{x}_k)$$

This works because the direction of $-\nabla f(\mathbf{x}_k)$ and $-\nabla f(\mathbf{x}_k) / \|\nabla f(\mathbf{x}_k)\|$ are exactly the same.

The latter has the disadvantage of requiring division by small number (bad for numerical stability); remember that ultimately we want $\nabla f(\mathbf{x}_k) \rightarrow 0$ so $\|\nabla f(\mathbf{x}_k)\|$ is expected to be small near the final steps of the algorithm.

The resulting algorithm is the **steepest descent** or **gradient descent** algorithm.

Algorithm 2 Steepest descent

Require: $\mathbf{x}_0 \in \Omega$, tolerance $\varepsilon > 0$ small

```
procedure DESCENT( $\mathbf{x}_0, \varepsilon$ )  
   $\mathbf{x} \leftarrow \mathbf{x}_0$   
  while  $\|\nabla f(\mathbf{x})\| > \varepsilon$  do  
     $\mathbf{p} \leftarrow -\nabla f(\mathbf{x})$   
    find step size  $\alpha > 0$   
     $\mathbf{x} \leftarrow \mathbf{x} + \alpha \mathbf{p}$   
  end while  
  return  $\mathbf{x}$   
end procedure
```

We will discuss how to find α to guarantee convergence to a stationary point.

Using the quadratic approximation, we find $\mathbf{p}_k$ by solving

$$\min_{\mathbf{p} \in \mathbb{R}^n} \left(f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top \mathbf{p} + \frac{1}{2} \mathbf{p}^\top \nabla^2 f(\mathbf{x}_k) \mathbf{p} \right).$$

What value of $\mathbf{p}$ will minimize this?

Let's define a new function $m(\mathbf{p}) = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top \mathbf{p} + \frac{1}{2} \mathbf{p}^\top \nabla^2 f(\mathbf{x}_k) \mathbf{p}$.

So we want to find $\min_{\mathbf{p}} m(\mathbf{p})$.

We can use the second-order condition for optimality to find the minimum:

$\nabla m(\mathbf{p}) = 0$ and $\nabla^2 m(\mathbf{p}) \succ 0$.

$$m(\mathbf{p}) = f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top \mathbf{p} + \frac{1}{2} \mathbf{p}^\top \nabla^2 f(\mathbf{x}_k) \mathbf{p}$$

I'll rewrite this even further: define $\mathbf{a} = \nabla f(\mathbf{x}_k)$ and $\mathbf{B} = \nabla^2 f(\mathbf{x}_k)$.

$$m(\mathbf{p}) = f(\mathbf{x}_k) + \mathbf{a}^\top \mathbf{p} + \frac{1}{2} \mathbf{p}^\top \mathbf{B} \mathbf{p}$$

Now we find the derivative of $m(\mathbf{p})$ with respect to $\mathbf{p}$. We've calculated the derivatives of linear and quadratic terms like these back in lecture 6:

$$[Dm(\mathbf{p})](\mathbf{h}) = \mathbf{a}^\top \mathbf{h} + \mathbf{p}^\top \mathbf{B} \mathbf{h}$$

$$[Dm(\mathbf{p})](\mathbf{h}) = \mathbf{a}^\top \mathbf{h} + \mathbf{p}^\top \mathbf{B} \mathbf{h}$$

So finally the gradient of $m(\mathbf{p})$ is

$$\nabla m(\mathbf{p}) = \mathbf{a} + \mathbf{B} \mathbf{p} = \nabla f(\mathbf{x}_k) + \nabla^2 f(\mathbf{x}_k) \mathbf{p}$$

The Hessian of $m(\mathbf{p})$ is $\mathbf{B} = \nabla^2 f(\mathbf{x}_k)$.

OK, back to finding the minimum of $m(\mathbf{p})$. By the second-order condition for optimality, we need to solve $\nabla m(\mathbf{p}) = 0$ and $\nabla^2 m(\mathbf{p}) \succ 0$.

Setting the gradient to 0, we get

$$\nabla f(\mathbf{x}_k) + \nabla^2 f(\mathbf{x}_k) \mathbf{p} = 0$$

If $\nabla^2 f(\mathbf{x}_k) \succ 0$ (e.g., if f strictly convex), then we know that $\nabla^2 f(\mathbf{x}_k)$ has an inverse, so we can solve for $\mathbf{p}$ to get the minimizer:

$$\mathbf{p}_k = -(\nabla^2 f(\mathbf{x}_k))^{-1} \nabla f(\mathbf{x}_k)$$

This direction is called the **Newton direction**.

Definition (Descent direction)

If the search direction satisfies

$$f(\mathbf{x}_k + \alpha \mathbf{p}_k) < f(\mathbf{x}_k)$$

for some $\alpha > 0$ then $\mathbf{p}_k$ is called a **descent direction**.

Algorithms in optimization are generally based on one principle: finding a descent direction and taking a step in that direction.

At the next iteration, the value of the function decreases.

Ideally, the algorithm will reach a local minimum.

Here is a different definition of descent direction: if the dot product of the gradient and a direction is negative, then the direction is a descent direction.

This is taken by some optimization specialists to be the definition of descent direction; our definition is more general. In particular, the converse holds if f is convex.

Theorem

Let $f : \Omega \subset \mathbb{R}^n \rightarrow \mathbb{R}$ be C^1 . If $\nabla f(\mathbf{x}_k)^\top \mathbf{p}_k < 0$, then there is some $\alpha > 0$ such that $f(\mathbf{x}_k + \alpha \mathbf{p}_k) < f(\mathbf{x}_k)$.

Proof of Theorem 2.1

If $\nabla f(\mathbf{x}_k)^\top \mathbf{p}_k < 0$, the continuity of ∇f implies that $\nabla f(\mathbf{x}_k + \alpha \mathbf{p}_k)^\top \mathbf{p}_k < 0$ for sufficiently small α .

For all such α , Taylor's theorem furnishes $\xi \in (0, 1)$ such that

$$f(\mathbf{x}_k + \alpha \mathbf{p}_k) = f(\mathbf{x}_k) + \alpha \nabla f(\mathbf{x}_k + \xi \alpha \mathbf{p}_k)^\top \mathbf{p}_k < f(\mathbf{x}_k)$$

This theorem is the converse of the previous one. It says that if the function is convex and the movement direction leads to a decrease in the function value, then the gradient and the movement direction must be pointing in opposite directions.

Theorem

If f is convex and there exists $\alpha > 0$ such that $f(\mathbf{x}_k + \alpha \mathbf{p}_k) < f(\mathbf{x}_k)$, then $\nabla f(\mathbf{x}_k)^\top \mathbf{p}_k < 0$.

By the first order condition for convexity,

$$f(\mathbf{x}_k + \alpha \mathbf{p}_k) \geq f(\mathbf{x}_k) + \alpha \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k$$

Rearranging,

$$\nabla f(\mathbf{x}_k)^\top \mathbf{p}_k \leq \frac{f(\mathbf{x}_k + \alpha \mathbf{p}_k) - f(\mathbf{x}_k)}{\alpha} < 0$$

Common shorthand in optimization:

$$f_k = f(\mathbf{x}_k), \quad \mathbf{g}_k = \nabla f(\mathbf{x}_k), \quad H_k = \nabla^2 f(\mathbf{x}_k)$$

If the Hessian is positive definite, then the Newton direction is a descent direction, and the steepest descent direction is also always a descent direction.

Theorem

1. *If $\nabla^2 f(\mathbf{x}_k) \succ 0$, then the Newton step $\mathbf{p}_k = -\nabla^2 f(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k)$ is a descent direction.*
2. *The steepest descent direction, $\mathbf{p}_k = -\nabla f(\mathbf{x}_k)$, is always a descent direction.*

Proof. By Theorem 2.1, if $\mathbf{g}_k^\top \mathbf{p}_k < 0$, then $\mathbf{p}_k$ is a descent direction:

1. $\nabla f(\mathbf{x}_k)^\top \mathbf{p}_k = \mathbf{g}_k^\top \mathbf{p}_k = -\mathbf{g}_k^\top H_k^{-1} \mathbf{g}_k < 0$
2. $\nabla f(\mathbf{x}_k)^\top \mathbf{p}_k = -\mathbf{g}_k^\top \mathbf{g}_k = -\|\mathbf{g}_k\|^2 < 0.$

When the Newton direction isn't a descent direction

In general we will not have $\nabla^2 f(\mathbf{x}_k) \succ 0$ for all $\mathbf{x}_k$, so the Newton direction may not be a descent direction.

In this case, we find $\mathbf{p}_k$ such that $\mathbf{p}_k^\top \nabla^2 f(\mathbf{x}_k)^{-1} \mathbf{p}_k < 0$ (direction of negative curvature).

If $\nabla^2 f(\mathbf{x}_k)$ has a positive eigenvalue, we may pick $\mathbf{p}_k$ to be in any of the eigenspaces corresponding to positive eigenvalues.

For strictly convex C^2 functions, however, the Newton direction will always be a descent direction.

Finding step size

We have seen two descent directions:

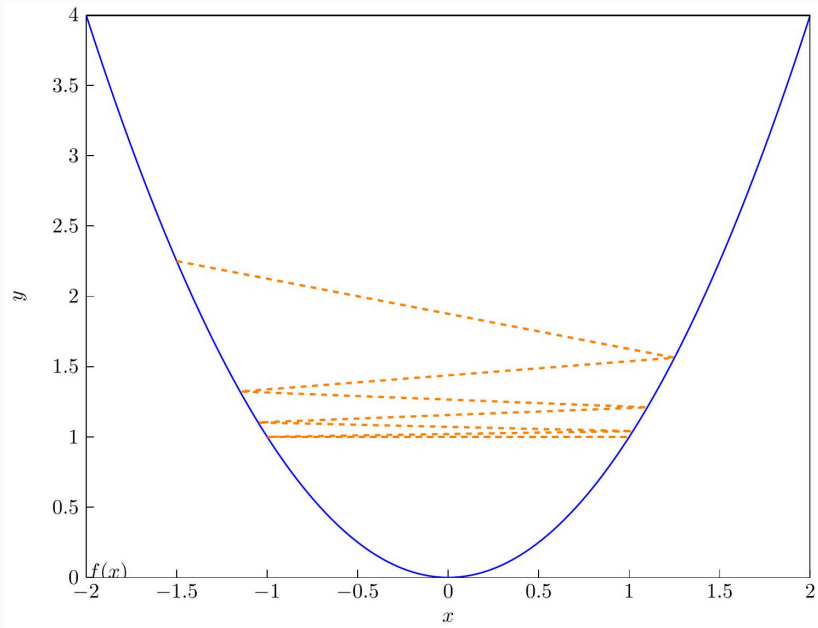
$$\mathbf{p}_k = -\mathbf{g}_k, \quad \mathbf{p}_k = -H_k^{-1} \mathbf{g}_k$$

How do we find α_k ?

The harder part is finding $\mathbf{p}_k$, actually; finding α_k is not too expensive.

(Aside: for Newton's method we can take $\alpha_k \equiv 1$.)

By the way, in modern machine learning parlance, step size α_k is called the learning rate.



Value of f is reduced at every step and yet does not converge to a minimizer. The problem with is that each step achieves too little reduction relative to the step size.

Why is the choice of α_k important? The descent direction only guarantees

Definition (Wolfe Conditions)

Definitions 13.1 (Wolfe conditions). A step size $\alpha_k > 0$ satisfies the Wolfe conditions if for some $0 < c_1 < c_2 < 1$, the following inequalities hold:

Armijo rule:

$$f(\mathbf{x}_k + \alpha_k \mathbf{p}_k) \leq f(\mathbf{x}_k) + c_1 \alpha_k \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k$$

Curvature condition:

$$\nabla f(\mathbf{x}_k + \alpha_k \mathbf{p}_k)^\top \mathbf{p}_k \geq c_2 \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k$$

The **strong Wolfe conditions** require α_k to satisfy the (1) and

$$\left| \nabla f(\mathbf{x}_k + \alpha_k \mathbf{p}_k)^\top \mathbf{p}_k \right| \leq c_2 \left| \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k \right|$$

The Armijo condition guarantees sufficient decrease, i.e., $f(\mathbf{x}_{k+1})$ is sufficiently smaller than $f(\mathbf{x}_k)$. The curvature condition guarantees reasonable progress, i.e. $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$ is reasonably far from $\mathbf{x}_k$ (i.e. α_k is not too small).

Although the Wolfe conditions seem restrictive, the following theorem gives reasonable conditions under which the Wolfe conditions are satisfied.

Theorem

Suppose

1. $f \in C^1(\mathbb{R}^n)$
2. $\nabla f(\mathbf{x}_k)^\top \mathbf{p}_k < 0$
3. $g(\alpha) = f(\mathbf{x}_k + \alpha \mathbf{p}_k)$ is bounded below for $\alpha > 0$
4. c_1 and c_2 any constant with $0 < c_1 < c_2 < 1$.

Then there exists $\alpha_k > 0$ satisfying the Wolfe conditions and strong Wolfe conditions.

Note that $g(\alpha) = f(\mathbf{x}_k + \alpha \mathbf{p}_k)$ is bounded below and the line $\ell(\alpha) = f(\mathbf{x}_k) + \alpha c_1 \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k$ is unbounded below. Since

$$g'(0) = \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k < c_1 \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k = \ell'(0) < 0,$$

g decreases faster than ℓ in a neighborhood of 0, i.e. the graph of g lies below graph of ℓ . Then there exists a smallest $\tilde{\alpha} > 0$ such that $g(\tilde{\alpha}) = \ell(\tilde{\alpha})$, i.e.

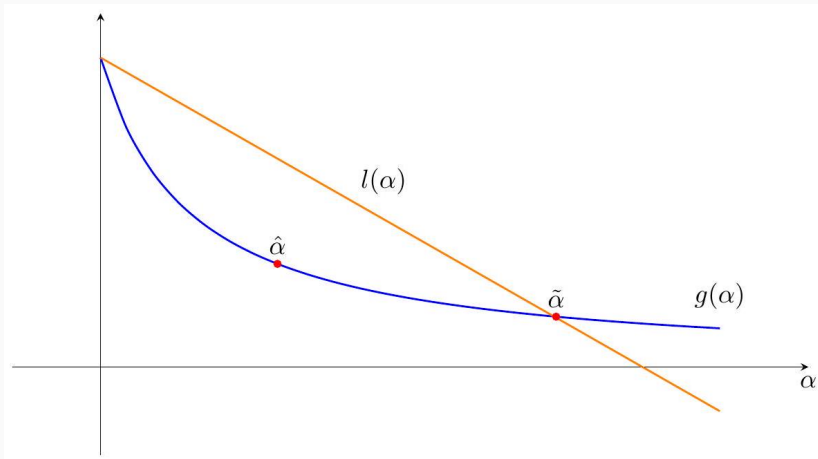
$$f(\mathbf{x}_k + \tilde{\alpha} \mathbf{p}_k) = f(\mathbf{x}_k) + \tilde{\alpha} c_1 \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k$$

The Armijo condition clearly holds for all $\alpha < \tilde{\alpha}$. By the mean value theorem, there exists $\hat{\alpha} \in (0, \tilde{\alpha})$ such that

$$g'(\hat{\alpha}) = \frac{g(\tilde{\alpha}) - g(0)}{\tilde{\alpha} - 0}.$$

By the chain rule,

$$f(\mathbf{x}_k + \tilde{\alpha} \mathbf{p}_k) - f(\mathbf{x}_k) = \tilde{\alpha} \underbrace{\nabla f(\mathbf{x}_k + \hat{\alpha} \mathbf{p}_k)^\top}_{g'(\hat{\alpha})} \mathbf{p}_k$$



The curvature condition in the Wolfe conditions is

$$\nabla f(\mathbf{x}_k + \alpha \mathbf{p}_k)^\top \mathbf{p}_k > c_2 \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k,$$

whereas the curvature condition in the strong Wolfe conditions is

$$\left| \nabla f(\mathbf{x}_k + \alpha \mathbf{p}_k)^\top \mathbf{p}_k \right| < c_2 \left| \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k \right|.$$

Note that these are the same if

$$0 > \nabla f(\mathbf{x}_k + \alpha \mathbf{p}_k)^\top \mathbf{p}_k > c_2 \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k$$

but can be different if

$$\nabla f(\mathbf{x}_k + \alpha \mathbf{p}_k)^\top \mathbf{p}_k > 0 > c_2 \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k$$

Corollary

If α_k satisfies Wolfe (resp. strong Wolfe) with strict inequality, then there exists $\varepsilon > 0$ such that all $\alpha \in (\alpha_k - \varepsilon, \alpha_k + \varepsilon)$ satisfy Wolfe (resp. strong Wolfe) with strict inequality. Proof. Since $f \in C^1$, both f and ∇f are continuous. Therefore, so are

$$F(\alpha) := f(\mathbf{x}_k) + c_1 \alpha \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k - f(\mathbf{x}_k + \alpha \mathbf{p}_k), \quad G(\alpha) := \nabla f(\mathbf{x}_k + \alpha \mathbf{p}_k)^\top \mathbf{p}_k - c_2 \nabla f(\mathbf{x}_k)$$

Since $F(\alpha_k) > 0$ and $G(\alpha_k) > 0$, there exists $\varepsilon > 0$ such that $F(\alpha) > 0$ and $G(\alpha) > 0$ in an ε -neighborhood of α_k .

Theorem

If

1. $f \in C^1(\mathbb{R}^n)$ and $\nabla f \in \text{Lip}_\gamma(\mathbb{R}^n)$
2. f bounded below
3. $\mathbf{p}_k$ and α_k satisfy Wolfe's condition then $\lim_{k \rightarrow \infty} \nabla f(\mathbf{x}_k) = \mathbf{0}$. *Proof. Defer to the lecture after next.*

In optimization, the Lipschitz condition typically appears in the derivative; this is a stronger condition than $f \in C^1$ but weaker than $f \in C^2$. Secondly, this theorem only guarantees that the sequences of gradients converges. In particular, $(\mathbf{x}_k)_{k=1}^{\infty}$ does not necessarily converge.

- Not good enough: $f(\mathbf{x}_k)$ is decreasing
- Better: $\nabla f(\mathbf{x}_k) \rightarrow \mathbf{0}$
- Even better: $\mathbf{x}_k \rightarrow \mathbf{x}_*$, where $\nabla f(\mathbf{x}_*) = \mathbf{0}$
- Best: $\mathbf{x}_k \rightarrow \mathbf{x}_*$, where $\nabla f(\mathbf{x}_*) = \mathbf{0}$ and $\nabla^2 f(\mathbf{x}_*) \succeq 0$. ‘Best’ and ‘even better’ are the same for convex function. In this course, we will be content with $\nabla f(\mathbf{x}_k) \rightarrow \mathbf{0}$.

Recall the two questions we have been discussing.

1. How do we find $\mathbf{p}_k$
2. How do we find α_k ?

We can answer item 2 with a line search. Recall that getting to $\mathbf{x}_{k+1}$ from $\mathbf{x}_k$ requires knowing which direction to take ($\mathbf{p}_k$) and knowing how much to step in that direction (α_k).

If $\min f(\mathbf{x}) = f(\mathbf{x}_*)$, then $\operatorname{argmin} f(\mathbf{x}) = \mathbf{x}_*$. Typically argmin is a set of global minimizers, but we will occasionally abuse notation. Here, we discuss two methods to compute α_k at each iteration.

Exact line search: find the best α along the ray $\{\mathbf{x}_k + \alpha \mathbf{p}_k : \alpha \geq 0\}$ in the sense of

$$\alpha_k = \operatorname{argmin}_{\alpha \geq 0} f(\mathbf{x}_k + \alpha \mathbf{p}_k).$$

If we minimize $g(\alpha)$, then minimizer α_* is the step size α_k . However, there is a problem associated with solving a univariate optimization problem every step: it can be very expensive. We use exact line search only when the univariate minimization is computationally cheaper than finding $\mathbf{p}_k$ (it is better to spend energy finding the right direction).

Backtracking line search

By Corollary 3.1, we expect a nonzero probability of finding an α_k that satisfies the Wolfe conditions if we simply pick it at random. Backtracking line search is a slightly more principled way to do this. Start by guessing that $\alpha_k = 1$, which often already satisfies Wolfe conditions; if not, then we reduce it by a fraction $\rho \in (0, 1)$ and check again. Repeat until we find one that satisfies Armijo condition. We may repeat the procedure again until we find one that also satisfies the curvature condition.

Algorithm 3 Backtracking line search (BTLS)

Require: $\mathbf{p}_k \in \mathbb{R}^n, \mathbf{x}_k \in \mathbb{R}^n, c \in (0, 1), \rho \in (0, 1)$

procedure BTLS($\mathbf{p}_k, x_k, c, \rho$)

$\alpha \leftarrow 1$

while $f(\mathbf{x}_k + \alpha \mathbf{p}_k) > f(\mathbf{x}_k) + c\alpha \nabla f(\mathbf{x}_k)^\top \mathbf{p}_k$ do // Armijo not satisfied

$\alpha \leftarrow \rho \alpha$

end while

$\alpha_k = \alpha$ // this is the best step size

return α_k

end procedure

Barzilai-Borwein step size

One may avoid line searches entirely by using a fixed formula for step size:

$$\alpha_{k+1} = \frac{\left| (\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k))^{\top} (\mathbf{x}_{k+1} - \mathbf{x}_k) \right|}{\|\nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)\|^2}.$$

This has become popular recently when used in conjunction with stochastic gradient descent. We will discuss it later after we discussed quasi-Newton methods. Note that this is a two-step method, unlike line searches, it requires us to store two successive gradients.

With these tools, we can give the complete algorithm for steepest descent.

Algorithm 4 Steepest descent with Barzilai–Borwein step size

Require: $\mathbf{x}_0 \in \mathbb{R}^n, \varepsilon > 0$ (small)

procedure SDBTLS($\mathbf{x}_0, \varepsilon$)

$k \leftarrow 0$

 repeat

 Descent direction: $\mathbf{p}_k \leftarrow -\nabla f(\mathbf{x}_k)$

 Step size: $\alpha_k \leftarrow |(\mathbf{p}_k - \mathbf{p}_{k-1})^\top (\mathbf{x}_k - \mathbf{x}_{k-1})| / \|\mathbf{p}_k - \mathbf{p}_{k-1}\|^2$

 Update: $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k \mathbf{p}_k$; $k \leftarrow k + 1$

 until Stopping condition: $\|\nabla f(\mathbf{x}_k)\| < \varepsilon$.

$\mathbf{x}_* \leftarrow \mathbf{x}_k$

 return $\mathbf{x}_*$

end procedure

Alternatively the step size can be determined through exact or backtracking line search.